



Proyecto Final Integrador

Plataforma de Datos para DataCommerce GT

Informe Técnico de Ingeniería de Datos

Curso: Ingeniería de Datos

Equipo: Grupo 2

Fecha: Julio 2026

Integrantes: Luis Escobar - Edwin Fernandez - Gerardo Carranza

Tecnologías utilizadas: Python, Pandas, SQLite, Google BigQuery, SQL, Apache Airflow, Docker, Matplotlib y Seaborn.

Repositorio:

https://github.com/edwinfernandez2312-hue/proyecto_final_grupo2

Índice

1. Resumen ejecutivo técnico
 2. Contexto del negocio
 3. Problema empresarial
 4. Objetivos del proyecto
 5. Alcance de la solución
 6. Arquitectura de la solución
 7. Fuentes de datos utilizadas
 8. Modelo relacional conceptual
 9. Reglas de calidad de datos aplicadas
 10. Modelo dimensional tipo galaxia
 11. Proceso ETL implementado
 12. Carga del Data Warehouse local y BigQuery
 13. Consultas SQL analíticas
 14. Indicadores construidos con Python y Pandas
 15. Visualizaciones y conclusiones de negocio
 16. Diccionario de datos
 17. Orquestación con Apache Airflow y Docker
 18. Evidencia de ejecución
 19. Reproducibilidad del proyecto
 20. Seguridad y control de versiones
 21. Limitaciones
 22. Conclusiones técnicas
 23. Anexos
-

1. Resumen ejecutivo técnico

El presente proyecto desarrolla una plataforma de datos para la empresa ficticia **DataCommerce GT**, dedicada a la comercialización de productos tecnológicos mediante distintos canales de venta. La solución implementada integra información proveniente de archivos CSV, Excel, JSON, una base SQLite y una API REST simulada, con el objetivo de centralizar los datos operativos y convertirlos en información analítica confiable.

La plataforma fue construida bajo un flujo de ingeniería de datos completo: extracción de fuentes heterogéneas, limpieza y transformación con Python y Pandas, construcción de un modelo relacional conceptual, diseño de un modelo dimensional tipo galaxia, carga de un Data Warehouse local en SQLite, migración hacia Google BigQuery, ejecución de consultas SQL analíticas, cálculo de KPIs con Pandas y generación automática de visualizaciones.

Adicionalmente, el pipeline fue orquestado mediante **Apache Airflow** ejecutado en **Docker**, permitiendo reproducibilidad, ejecución controlada y trazabilidad del proceso. La solución demuestra una arquitectura funcional orientada a un entorno empresarial, donde la información puede ser procesada nuevamente sin intervención manual y utilizada como base para análisis gerencial.

2. Contexto del negocio

DataCommerce GT es una empresa ficticia dedicada a la venta de productos tecnológicos. Su operación comercial se realiza mediante distintos canales:

Canal	Descripción
Tienda física	Ventas realizadas en puntos comerciales
Comercio electrónico	Ventas realizadas mediante canal digital
Ventas corporativas	Ventas dirigidas a clientes empresariales
WhatsApp	Canal de atención y venta directa

Con el crecimiento de la operación, la información comenzó a almacenarse en diferentes fuentes y formatos. Esta situación generó una fragmentación de los datos, dificultando que la gerencia obtuviera reportes confiables, oportunos y consistentes.

El proyecto simula un escenario real de consultoría en el cual se requiere diseñar e implementar una solución de ingeniería de datos capaz de integrar múltiples fuentes, limpiar

la información, centralizar en un Data Warehouse y generar indicadores para apoyar la toma de decisiones.

3. Problema empresarial

El principal problema de DataCommerce GT no es la ausencia de información, sino la falta de una **fuentes única de verdad**. Cada área o canal puede manejar sus propios archivos, formatos y reportes, lo que provoca diferencias entre los resultados generados por distintos departamentos.

Los problemas identificados en el contexto del proyecto son:

Problema	Impacto en el negocio
Datos dispersos en múltiples fuentes	Dificulta la consolidación de reportes
Formatos distintos entre archivos	Complica la integración automática
Fechas inconsistentes	Afecta el análisis temporal
Campos vacíos o incompletos	Reduce la calidad analítica
Registros duplicados	Puede inflar ventas, clientes o inventario
Falta de centralización	Genera reportes no comparables
Métricas aisladas	Impide relacionar ventas, inventario y marketing

La dirección necesita una plataforma que centralice la información, aplique reglas de calidad, permite consultas analíticas y genere indicadores confiables para la toma de decisiones.

4. Objetivos del proyecto

4.1 Objetivo general

Construir una solución completa de ingeniería de datos que integre información de múltiples fuentes, la transforme, la almacene en un Data Warehouse y genere indicadores útiles para apoyar la toma de decisiones gerenciales.

4.2 Objetivos específicos

Objetivo específico	Cumplimiento en la solución
Integrar archivos CSV, Excel, JSON, SQLite y API REST simulada	Se integraron ventas, productos, clientes, inventario y marketing
Diseñar un modelo relacional	Se definió un modelo conceptual con entidades operativas
Diseñar un modelo dimensional	Se implementó un modelo tipo galaxia con dimensiones y hechos
Implementar un proceso ETL reproducible	Se desarrolló un pipeline modular en Python
Aplicar reglas de calidad de datos	Se limpiaron duplicados, fechas, nulos, tipos y campos clave
Cargar la información al Data Warehouse	Se generó un DW local en SQLite
Migrar información a la nube	Se cargaron las tablas a Google BigQuery
Ejecutar consultas SQL analíticas	Se implementaron 7 consultas analíticas
Construir KPIs con Python y Pandas	Se calcularon métricas comerciales, financieras y operativas
Crear visualizaciones	Se generaron gráficos PNG automáticamente
Orquestar el proceso	Se utilizó Apache Airflow sobre Docker

5. Alcance de la solución

El presente informe documenta la solución técnica desarrollada para el proyecto final integrador. El alcance incluye:

Área	Incluido en el proyecto
Extracción	Lectura de fuentes CSV, Excel, JSON, SQLite y API simulada
Transformación	Limpieza, estandarización y calidad con Python/Pandas
Modelado	Modelo relacional conceptual y modelo dimensional tipo galaxia
Almacenamiento	Data Warehouse local

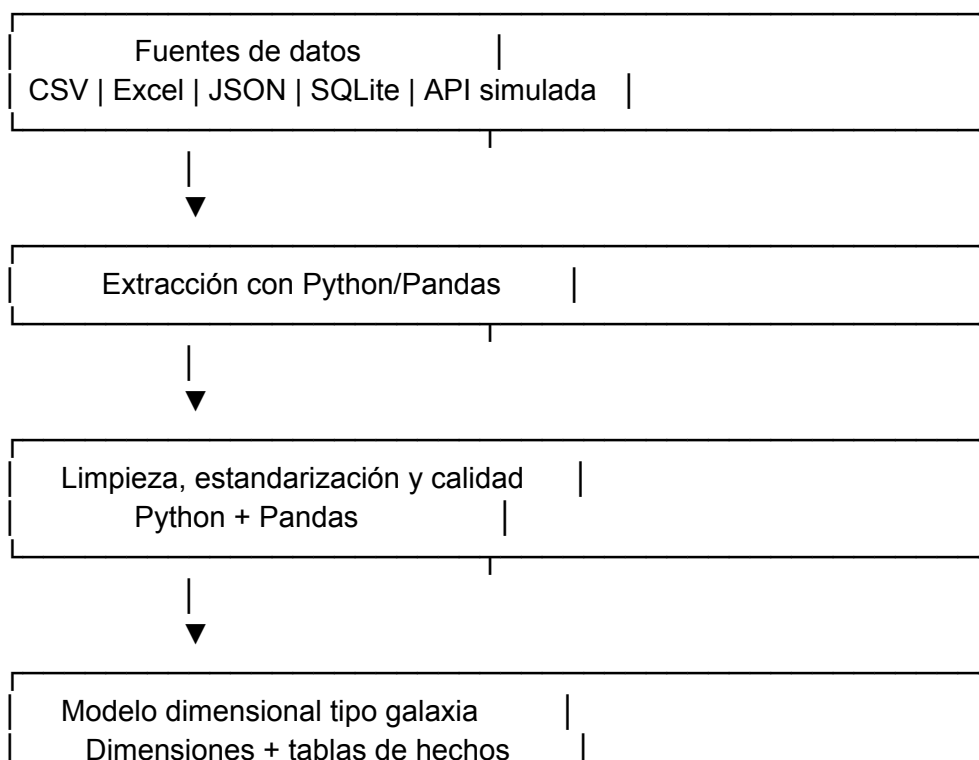
Nube	Migración de tablas hacia Google BigQuery
SQL	Consultas analíticas ejecutadas sobre BigQuery
KPIs	Indicadores generados con Python y Pandas
Visualización	Gráficos exportados en PNG
Orquestación	Git Actions y DAG ejecutado en Apache Airflow con Docker
Documentación	Estructura técnica, diccionario de datos y evidencias

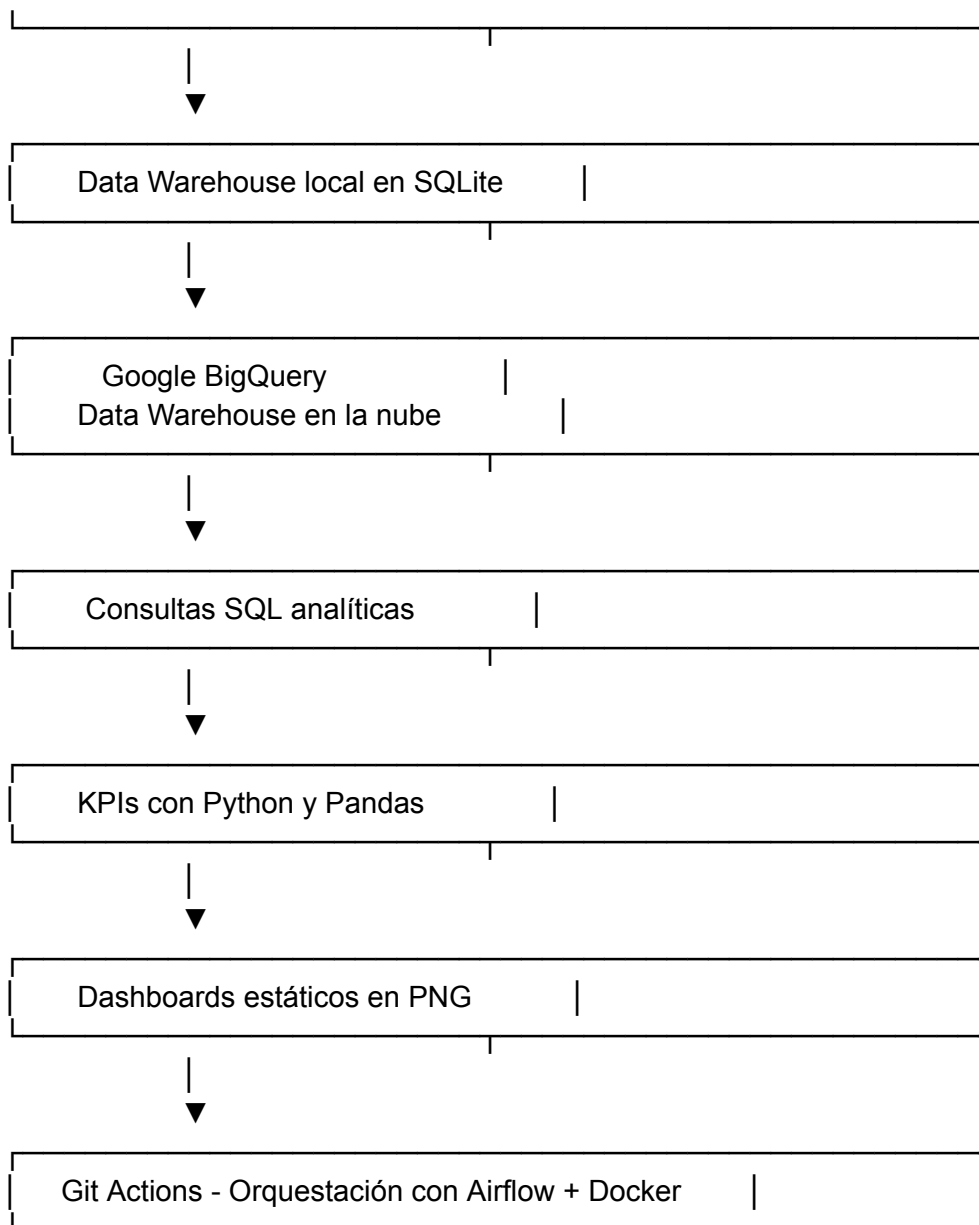
El análisis empresarial profundo se considera un entregable complementario, ya que el conjunto de datos utilizado es una muestra simulada y limitada. Este documento se centra en demostrar la arquitectura, funcionamiento, trazabilidad y reproducibilidad de la plataforma de datos.

6. Arquitectura de la solución

La arquitectura se diseñó bajo un flujo de procesamiento por etapas. Cada componente cumple una función específica dentro del pipeline, desde la lectura de fuentes hasta la generación de indicadores.

6.1 Flujo general de arquitectura





6.2 Componentes técnicos

Componente	Tecnología	Función
Extracción	Python / Pandas	Leer datos desde distintas fuentes
Transformación	Pandas	Limpiar, normalizar y preparar datos
Data Warehouse local	SQLite	Persistir dimensiones y hechos localmente
Data Warehouse en nube	Google BigQuery	Centralizar tablas analíticas en la nube

Consultas analíticas	SQL	Obtener DataFrames analíticos desde BigQuery
KPIs	Python / Pandas / NumPy	Calcular métricas de negocio
Visualización	Matplotlib / Seaborn	Generar gráficos exportables
Orquestación	Apache Airflow	Automatizar la ejecución del pipeline
Contenedores	Docker Compose	Ejecutar Airflow y dependencias

7. Fuentes de datos utilizadas

El proyecto integra cinco fuentes que representan áreas operativas de DataCommerce GT.

Fuente	Formato	Archivo / origen	Uso dentro del pipeline
Ventas	CSV	<code>ventas.csv</code>	Registra las transacciones comerciales
Productos	Excel	<code>productos.xlsx</code>	Contiene el catálogo de productos
Clientes	JSON	<code>clientes.json</code>	Almacena información de clientes
Inventario	SQLite	<code>inventario.db</code>	Contiene existencias y movimientos
Marketing	JSON / API simulada	<code>api_marketing_response.json</code>	Registra campañas, costos, leads y conversiones

Las fuentes originales se conservan en la carpeta `data/` y no se modifican manualmente. Todas las transformaciones se realizan mediante código, cumpliendo la regla de reproducibilidad del proyecto.

8. Modelo relacional conceptual

El modelo relacional conceptual representa la estructura operativa del negocio antes de ser transformada en un modelo analítico. Este modelo permite identificar las entidades principales y sus relaciones lógicas.

8.1 Entidades principales

Entidad	Descripción
Cliente	Persona o empresa que realiza compras
Producto	Artículo tecnológico comercializado por la empresa
Venta	Transacción comercial realizada por un cliente
Inventario	Registro de existencias por producto y bodega
Movimiento de inventario	Entradas o salidas de productos
Campaña de marketing	Acción publicitaria asociada a métricas digitales
Canal de venta	Medio por el cual se realiza una venta
Sucursal	Punto comercial asociado a la venta
Método de pago	Forma de pago utilizada en una transacción

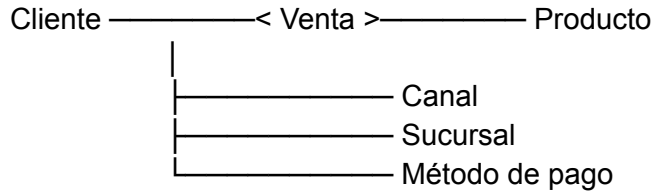
8.2 Relaciones conceptuales

Relación	Cardinalidad lógica
Cliente - Venta	Un cliente puede realizar muchas ventas
Producto - Venta	Un producto puede aparecer en muchas ventas
Producto - Inventario	Un producto puede tener inventario en una o varias bodegas
Producto - Movimiento de inventario	Un producto puede tener múltiples entradas o salidas
Venta - Canal	Una venta pertenece a un canal de venta
Venta - Sucursal	Una venta puede estar asociada a una sucursal
Venta - Método de pago	Una venta utiliza un método de pago
Campaña - Marketing	Una campaña registra métricas como costo, leads y conversiones

Tiempo - Hechos

Las ventas, movimientos y campañas se analizan por fecha

8.3 Representación conceptual



Producto ———< Inventario >——— Bodega

Producto ———< Movimiento de inventario

Campaña ———< Marketing

Tiempo ———< Ventas / Movimientos / Marketing

Este modelo relacional sirve como base conceptual para comprender cómo se conectan las entidades operativas antes de pasar al diseño dimensional.

9. Reglas de calidad de datos aplicadas

Las reglas de calidad fueron implementadas mediante código en los módulos de transformación. Se aplicaron reglas para eliminar duplicados, corregir formatos, normalizar texto, convertir tipos de datos, manejar valores nulos y proteger la integridad referencial.

9.1 Reglas aplicadas por fuente

Fuente	Regla aplicada	Justificación técnica
Cientes	Eliminación de duplicados por <code>cliente_id</code> , conservando el último registro	Evita duplicidad en la dimensión cliente
Cientes	Reemplazo de municipio vacío por " <code>No disponible</code> "	Permite conservar registros con información incompleta
Cientes	Conversión de <code>fecha_registro</code> a formato <code>YYYY-MM-DD</code>	Estandariza el análisis temporal
Cientes	Relleno de textos nulos con " <code>No disponible</code> "	Evita valores nulos en atributos descriptivos

Clientes	Conversión de edad a entero	Permite análisis demográfico consistente
Productos	Eliminación de duplicados por producto_id	Evita duplicar productos en el catálogo
Productos	Conversión de costo_unitario y precio_lista a numérico	Permite calcular margen y rentabilidad
Productos	Limpieza de espacios en columnas de texto	Homologa categorías y descripciones
Productos	Reemplazo de textos vacíos por " No disponible "	Evita campos descriptivos vacíos
Productos	Relleno de numéricos nulos con 0	Evita errores en operaciones matemáticas
Ventas	Eliminación de duplicados por venta_id	Evita inflar ingresos y cantidades
Ventas	Parseo de fechas en formatos %Y-%m-%d , %Y/%m/%d y %d/%m/%Y	Corrige inconsistencias de formato temporal
Ventas	Estandarización de canal_venta con formato título	Homologa nombres de canales
Ventas	Reemplazo de " Web " por " E-commerce "	Unifica el canal digital
Ventas	Relleno de metodo_pago con " No disponible "	Evita nulos en una dimensión analítica
Ventas	Conversión de cantidad , precio_unitario , descuento y total_venta a numérico	Permite cálculos comerciales y financieros
Ventas	Eliminación de registros sin venta_id , cliente_id , producto_id o fecha_venta	Protege la integridad referencial del modelo
Inventario	Eliminación de duplicados por producto_id y bodega	Evita duplicar existencias por ubicación
Inventario	Normalización de nombres de bodega	Homologa la dimensión bodega
Inventario	Conversión de existencia a entero	Permite análisis de stock

Movimientos	Eliminación de duplicados por <code>id</code>	Evita duplicar entradas o salidas
Movimientos	Conversión de fecha a <code>YYYY-MM-DD</code>	Permite integrarse con la dimensión tiempo
Movimientos	Normalización de tipo de movimiento	Homologa entradas y salidas
Movimientos	Conversión de <code>cantidad</code> a entero	Permite análisis operativo
Marketing	Renombrado de <code>campana_id</code> / <code>campana_id</code> a <code>campana_id</code>	Evita problemas por caracteres especiales
Marketing	Eliminación de duplicados por <code>campana_id</code>	Evita duplicar campañas
Marketing	Conversión de fecha a <code>YYYY-MM-DD</code>	Integra campañas con la dimensión tiempo
Marketing	Normalización de plataforma	Homologa nombres de plataformas
Marketing	Conversión de impresiones, clics, leads y conversiones a entero	Permite métricas digitales correctas
Marketing	Conversión de costo a numérico	Permite calcular ROAS y esfuerzo de marketing

Las reglas de ventas incluyen pareo de múltiples formatos de fecha, estandarización del canal, conversión de campos monetarios y eliminación de registros sin claves obligatorias. Las reglas de clientes convierten fechas, completan campos de texto y normalizan edad; las de productos convierten costos y precios a numéricos, limpian textos y rellenan valores nulos.

9.2 Justificación general

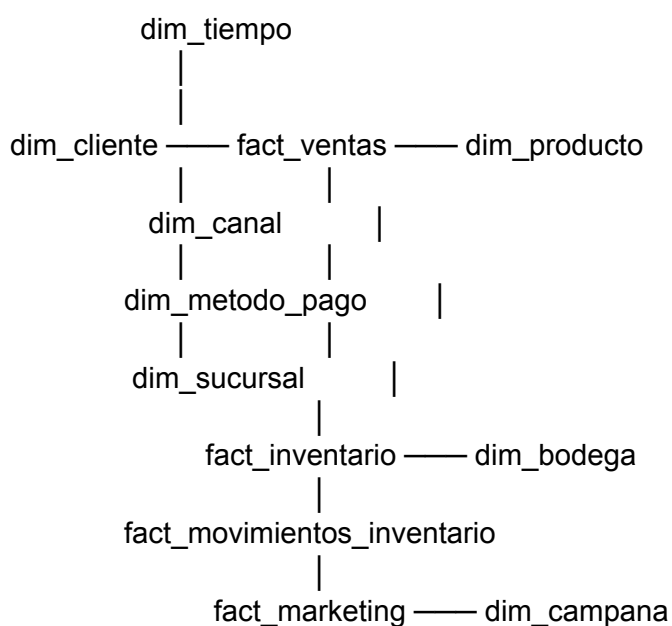
Estas reglas permiten que las fuentes heterogéneas puedan integrarse en un modelo común sin modificar manualmente los archivos originales. La limpieza garantiza que las tablas de dimensiones y hechos se construyen con datos consistentes, reduciendo errores en KPIs y consultas analíticas.

10. Modelo dimensional tipo galaxia

El modelo dimensional implementado corresponde a una **galaxia dimensional o constelación de hechos**, ya que no existe una única tabla de hechos central, sino varias tablas de hechos relacionadas con diferentes procesos del negocio.

10.1 Justificación del modelo galaxia

El negocio necesita analizar ventas, inventario, movimientos y marketing. Cada una de estas áreas genera eventos medibles diferentes, por lo que se construyeron varias tablas de hechos que comparten dimensiones comunes.



10.2 Dimensiones implementadas

Dimensión	Descripción	Clave principal
dim_tiempo	Calendario analítico con año, mes, día, trimestre, nombre de mes y día de semana	fecha_key
dim_cliente	Información descriptiva de clientes	cliente_key
dim_producto	Catálogo de productos	producto_key
dim_sucursal	Sucursales comerciales	sucursal_key
dim_canal	Canales de venta	canal_key

<code>dim_bodega</code>	Bodegas o ubicaciones de inventario	<code>bodega_key</code>
<code>dim_metodo_pago</code>	Métodos de pago	<code>metodo_pago_key</code>
<code>dim_campana</code>	Campañas y plataformas de marketing	<code>campana_key</code>

El módulo de construcción de dimensiones genera `dim_tiempo`, `dim_cliente`, `dim_producto`, `dim_sucursal`, `dim_canal`, `dim_bodega`, `dim_metodo_pago` y `dim_campana`, utilizando claves sustitutas para facilitar el análisis dimensional.

10.3 Tablas de hechos implementadas

Tabla de hechos	Proceso de negocio	Métricas principales
<code>fact_ventas</code>	Ventas comerciales	<code>cantidad</code> , <code>precio_unitario</code> , <code>descuento</code> , <code>total_venta</code>
<code>fact_inventario</code>	Estado de inventario	<code>existencia</code>
<code>fact_movimientos_inventario</code>	Entradas y salidas de inventario	<code>tipo_movimiento</code> , <code>cantidad</code>
<code>fact_marketing</code>	Campañas digitales	<code>impresiones</code> , <code>clics</code> , <code>costo</code> , <code>leads</code> , <code>conversiones</code>

La tabla `fact_ventas` conecta con dimensiones como tiempo, cliente, producto, sucursal, canal y método de pago; las tablas de inventario se relacionan con producto, bodega y tiempo; y la tabla de marketing se conecta con tiempo y campaña.

11. Proceso ETL implementado

El pipeline fue desarrollado de forma modular y coordinado desde una función principal que ejecuta todas las etapas del proceso.

11.1 Fase 1: Extracción

Fuente	Función	Descripción
Productos	<code>extraer_productos_excel()</code>	Lee el catálogo desde Excel

Ventas	<code>extraer_ventas_csv()</code>	Lee transacciones desde CSV
Clientes	<code>extraer_clientes_json()</code>	Lee datos de clientes desde JSON
Inventario	<code>extraer_inventario_db()</code>	Extrae existencias y movimientos desde SQLite
Marketing	<code>extraer_marketing_api()</code>	Obtiene campañas desde API simulada / JSON

11.2 Fase 2: Transformación

Fuente	Función	Propósito
Clientes	<code>transformar_clientes_json()</code>	Limpia duplicados, fechas, nulos y edad
Productos	<code>transformar_productos_excel()</code>	Normaliza costos, precios, textos y nulos
Ventas	<code>transformar_ventas_csv()</code>	Estandariza fechas, canales, pagos y métricas
Inventario	<code>transformar_inventario_db()</code>	Limpia existencias y movimientos
Marketing	<code>transformar_marketing_api()</code>	Normaliza campañas y métricas digitales

11.3 Fase 3: Construcción dimensional

Función	Resultado
<code>construir_todas_las_dimensiones()</code>	Genera las dimensiones del modelo
<code>construir_todas_las_tablas_hecho()</code>	Genera las tablas de hechos

11.4 Fase 4: Carga local

La carga local se realiza en SQLite mediante la función `cargar_data_warehouse()`. Esta función recibe los diccionarios de dimensiones y hechos, crea la ruta del Data

Warehouse y escribe cada DataFrame como tabla usando `to_sql()` con `if_exists="replace"`, lo que permite regenerar las tablas en cada ejecución sin duplicarlas.

```
(.venv) PS_C:\Users\edwin\PROYECTOS DATA\proyecto_final_grupo2\src> python main.py

[✓] Data Warehouse cargado en: C:\Users\edwin\PROYECTOS DATA\proyecto_final_grupo2\data\datacommerce_dw.db
Tablas creadas:
- dim_tiempo
- dim_cliente
- dim_producto
- dim_sucursal
- dim_canal
- dim_bodega
- dim_metodo_pago
- dim_campana
- fact_ventas
- fact_inventario
- fact_movimientos_inventario
- fact_marketing
```

11.5 Fase 5: Migración hacia BigQuery

La migración hacia BigQuery lee las tablas desde SQLite, valida que no estén vacías y las carga al dataset de BigQuery usando el cliente oficial de Google. La carga utiliza `WRITE_TRUNCATE`, por lo que reemplaza los datos existentes en cada ejecución, manteniendo un comportamiento reproducible.

11.6 Fase 6: Analítica

La capa analítica ejecuta consultas SQL almacenadas en la carpeta `sql/`. Cada consulta se ejecuta en BigQuery y devuelve un DataFrame de Pandas para alimentar la capa de KPIs.

11.7 Fase 7: Visualización

Finalmente, la capa de visualización genera gráficos estáticos en formato PNG, que se almacenan en la carpeta de reportes visuales.

12. Carga del Data Warehouse local y BigQuery

12.1 Data Warehouse local en SQLite

El Data Warehouse local se genera en el archivo:

data/datacommerce_dw.db

Este archivo contiene las dimensiones y hechos generados por el pipeline. La carga se realiza reemplazando las tablas existentes en cada ejecución, lo que evita duplicidad y permite ejecutar el pipeline varias veces con una salida consistente.

Tipo	Tablas cargadas
Dimensiones	<code>dim_tiempo</code> , <code>dim_cliente</code> , <code>dim_producto</code> , <code>dim_sucursal</code> , <code>dim_canal</code> , <code>dim_bodega</code> , <code>dim_metodo_pago</code> , <code>dim_campana</code>
Hechos	<code>fact_ventas</code> , <code>fact_inventario</code> , <code>fact_movimientos_inventario</code> , <code>fact_marketing</code>

12.2 Migración a Google BigQuery

Posteriormente, las tablas del Data Warehouse local son migradas hacia BigQuery. El proceso recorre una lista ordenada de tablas, lee cada una desde SQLite con Pandas y la carga a BigQuery mediante `load_table_from_dataframe()`. La configuración `WRITE_TRUNCATE` permite reemplazar la tabla destino en cada ejecución.

Este diseño permite combinar un entorno local de desarrollo con un Data Warehouse en la nube, facilitando pruebas locales y análisis escalables.

13. Consultas SQL analíticas

Las consultas SQL analíticas se almacenan como archivos independientes dentro de la carpeta `sql/`. El módulo de consultas lee cada archivo, lo ejecuta en BigQuery y devuelve los resultados como DataFrames de Pandas.

13.1 Portafolio de consultas

Archivo SQL	DataFrame generado	Propósito técnico
<code>01_rendimiento_comercial.sql</code>	<code>ventas_comercial</code>	Analiza ventas, ingresos, productos, canales y comportamiento comercial

02_conciliacion_inventario.sql	inventario_movimientos	Relaciona inventario actual con movimientos para evaluar stock y rotación
03_roi_marketing.sql	roi_marketing	Evalúa campañas, costos, leads, conversiones y retorno
04_top_clientes.sql	top_clientes	Identifica clientes de mayor valor mediante agregaciones de compra
05_ventas_vs_marketing.sql	ventas_marketing	Relaciona inversión de marketing con ingresos comerciales
06_rentabilidad_margen.sql	rentabilidad_margen	Calcula ingresos, costos, descuentos, ganancia y margen por producto
07_comportamiento_demografico.sql	demografia_canales	Analiza ingresos y pedidos por variables demográficas y canales

13.2 Explicación técnica resumida

Las consultas SQL cumplen la función de preparar vistas analíticas a partir del modelo dimensional cargado en BigQuery. En lugar de calcular todos los indicadores directamente sobre tablas crudas, las consultas consolidan hechos y dimensiones para generar DataFrames específicos por área de análisis: comercial, inventario, marketing, rentabilidad, clientes y demografía.

Este enfoque separa la lógica analítica en dos niveles:

Nivel	Herramienta	Función
SQL	BigQuery	Agregación, cruces y preparación analítica
Python/Pandas	Pandas / NumPy	Cálculo de KPIs, rankings, ratios y resúmenes

14. Indicadores contruidos con Python y Pandas

La capa de KPIs utiliza los DataFrames obtenidos desde BigQuery y aplica cálculos adicionales con Pandas y NumPy. El proceso genera métricas comerciales, financieras, operativas y ejecutivas.

14.1 KPIs comerciales y de marketing

KPI	Fórmula / lógica	Interpretación
<code>crecimiento_ingresos_pct</code>	Variación porcentual de ingresos mes a mes	Mide crecimiento o caída de ingresos
<code>roas</code>	$\text{ingresos_totales} / \text{inversion_total}$	Retorno sobre inversión publicitaria
<code>esfuerzo_marketing_pct</code>	$(\text{inversion_total} / \text{ingresos_totales}) * 100$	Porcentaje del ingreso destinado a marketing

14.2 KPIs de rentabilidad

KPI	Fórmula / lógica	Interpretación
<code>precio_promedio_venta</code>	$\text{ingresos_brutos} / \text{volumen_vendido}$	Ticket promedio por producto
<code>impacto_descuento_pct</code>	Descuento sobre ingreso bruto ajustado	Mide impacto comercial de descuentos
<code>contribucion_acumula_da_pct</code>	Acumulado de ganancia neta	Permite clasificación ABC
<code>categoria_rentabilidad</code>	Clasificación A/B/C por contribución	Segmenta productos por importancia financiera

14.3 KPIs de inventario

KPI	Fórmula / lógica	Interpretación
<code>ratio_salida_stock</code>	$\text{unidades_retiradas} / \text{stock_actual}$	Mide rotación relativa del inventario
<code>requiere_reabastecimiento_urgente</code>	Stock igual a 0 y salidas mayores a 0	Detecta riesgo operativo de agotamiento

14.4 Tarjetas ejecutivas de KPI

El resumen ejecutivo consolida métricas macro del negocio en un diccionario de KPI Cards. Entre los indicadores consolidados están ingresos totales del mes, ROAS actual, crecimiento mensual, ganancia neta total, margen promedio, productos agotados, cliente principal, canal estrella y método de pago favorito.

KPI ejecutivo	Resultado obtenido en ejecución
Ingresos totales del mes	75,992,953.78
ROAS actual	1,083.61
Crecimiento MoM	-0.68
Ganancia neta total	1,266,943,618.51
Margen promedio global	22.79%
Productos agotados	65
Cliente top 1	Gabriela Garcia
Canal estrella	E-commerce
Método de pago favorito	Tarjeta

Los valores anteriores corresponden a la ejecución del pipeline con el dataset de muestra. Los logs evidencian la ejecución de consultas SQL, cálculo de KPIs y generación de métricas ejecutivas.

15. Visualizaciones y conclusiones de negocio

La solución genera visualizaciones automáticas en formato PNG mediante Matplotlib y Seaborn. Cuando el pipeline se ejecuta dentro de Airflow, los gráficos se guardan en la ruta `/opt/airflow/reportes_visuales`.

15.1 Gráficos generados

Gráfico	Archivo	Descripción
---------	---------	-------------

Evolución de ingresos vs ROAS	01_tendencia_ventas_roas.png	Compara ingresos totales con retorno de marketing
Rentabilidad Top 5	02_rentabilidad_top5.png	Muestra productos con mayor ganancia neta
Distribución por canal	03_distribucion_canal.png	Presenta participación de ingresos por canal de venta
Ranking de clientes	04_ranking_clientes.png	Identifica clientes con mayor volumen de compra
Rotación de inventario	05_inventario_rotacion.png	Muestra productos con mayor salida relativa al stock

El módulo de gráficos genera visualizaciones sobre tendencia de ingresos y ROAS, rentabilidad por producto, distribución por canal, ranking de clientes e inventario, exportando cada gráfico como archivo PNG.

15.2 Conclusiones de negocio preliminares

A partir de los KPIs y visualizaciones generadas, se obtienen las siguientes conclusiones preliminares:

Área	Conclusión preliminar
Ventas	La plataforma permite consolidar ingresos totales y analizarlos por canal, cliente y producto
Marketing	El ROAS permite evaluar la relación entre inversión publicitaria e ingresos generados
Rentabilidad	La clasificación de productos permite identificar los artículos que más aportan a la ganancia neta
Clientes	El ranking de clientes ayuda a detectar compradores de mayor valor
Inventario	El ratio de salida sobre stock permite anticipar riesgos de agotamiento
Canales	La distribución por canal permite identificar el canal con mejor desempeño

Debido a que el dataset es una muestra simulada, estas conclusiones deben interpretarse como una validación funcional del pipeline y no como un diagnóstico definitivo de una empresa real.

16. Diccionario de datos

El diccionario de datos documenta el significado de las principales tablas y campos del modelo. Su propósito es facilitar la comprensión técnica del Data Warehouse y servir como referencia para consultas, análisis y mantenimiento.

16.1 Diccionario de tablas

Tabla	Tipo	Descripción
dim_tiempo	Dimensión	Calendario analítico para relacionar hechos por fecha
dim_cliente	Dimensión	Información descriptiva de clientes
dim_producto	Dimensión	Catálogo analítico de productos
dim_sucursal	Dimensión	Sucursales o puntos comerciales
dim_canal	Dimensión	Canales de venta utilizados
dim_bodega	Dimensión	Bodegas de inventario
dim_metodo_pago	Dimensión	Métodos de pago registrados
dim_campana	Dimensión	Campañas y plataformas de marketing
fact_ventas	Hecho	Métricas de ventas
fact_inventario	Hecho	Existencias actuales por producto y bodega
fact_movimientos_inventario	Hecho	Entradas y salidas de inventario
fact_marketing	Hecho	Métricas digitales de campañas

16.2 Campos principales por dimensión

Tabla	Campo	Descripción
dim_tiempo	fecha_key	Clave numérica de fecha en formato YYYYMMDD
dim_tiempo	fecha	Fecha en formato estándar
dim_tiempo	anio	Año de la fecha
dim_tiempo	mes	Mes numérico
dim_tiempo	trimestre	Trimestre del año
dim_cliente	cliente_key	Clave sustituta del cliente
dim_cliente	cliente_id_natural	Identificador original del cliente
dim_cliente	nombre	Nombre del cliente
dim_cliente	segmento_cliente	Segmento comercial del cliente
dim_producto	producto_key	Clave sustituta del producto
dim_producto	producto_id_natural	Identificador original del producto
dim_producto	nombre_producto	Nombre del producto
dim_producto	costo_unitario	Costo unitario del producto
dim_producto	precio_lista	Precio de lista del producto
dim_canal	canal_key	Clave sustituta del canal
dim_canal	canal_venta	Nombre del canal de venta

dim_campa na	campana_key	Clave sustituta de campaña
dim_campa na	campana_id_natu ral	Identificador original de campaña
dim_campa na	plataforma	Plataforma digital de la campaña

16.3 Campos principales por tabla de hechos

Tabla	Campo	Descripción
fact_ventas	venta_id	Identificador de la venta
fact_ventas	fecha_key	Clave de relación con dim_tiempo
fact_ventas	cliente_key	Clave de relación con dim_cliente
fact_ventas	producto_key	Clave de relación con dim_producto
fact_ventas	canal_key	Clave de relación con dim_canal
fact_ventas	metodo_pago_ key	Clave de relación con dim_metodo_pago
fact_ventas	cantidad	Cantidad vendida
fact_ventas	precio_unita rio	Precio unitario de venta
fact_ventas	descuento	Descuento aplicado
fact_ventas	total_venta	Valor total de la venta
fact_inventario	producto_key	Producto asociado al stock
fact_inventario	bodega_key	Bodega asociada al stock
fact_inventario	existencia	Cantidad disponible
fact_movimientos_inven tario	movimiento_i d	Identificador del movimiento
fact_movimientos_inven tario	tipo_movimie nto	Entrada o salida

<code>fact_movimientos_inventario</code>	<code>cantidad</code>	Unidades movidas
<code>fact_marketing</code>	<code>campana_key</code>	Campaña asociada
<code>fact_marketing</code>	<code>fecha_key</code>	Fecha de la campaña
<code>fact_marketing</code>	<code>impresiones</code>	Número de impresiones
<code>fact_marketing</code>	<code>clics</code>	Número de clics
<code>fact_marketing</code>	<code>costo</code>	Inversión de campaña
<code>fact_marketing</code>	<code>leads</code>	Leads generados
<code>fact_marketing</code>	<code>conversiones</code>	Conversiones obtenidas

17. Orquestación con Apache Airflow y Docker

La solución fue orquestada mediante Apache Airflow ejecutado dentro de un entorno Docker. El DAG principal ejecuta el pipeline completo mediante un `PythonOperator`, invocando la función principal del pipeline.

17.1 Configuración general

Elemento	Valor
DAG	<code>pipeline_datacommerce_etl_v1</code>
Operador	<code>PythonOperator</code>
Función ejecutada	<code>run_etl_pipeline()</code>
Frecuencia	<code>@daily</code>
Entorno	Docker Compose
Servicios	Webserver, Scheduler, Worker, Triggerer, PostgreSQL y Redis

17.2 Carpetas montadas en Docker

Carpeta local	Ruta dentro del contenedor	Uso
dags/	/opt/airflow/dags	DAGs de Airflow
src/	/opt/airflow/src	Código del pipeline
data/	/opt/airflow/data	Fuentes y Data Warehouse local
sql/	/opt/airflow/sql	Consultas analíticas
dashboards /	/opt/airflow/reportes_visuales	Reportes gráficos generados
logs/	/opt/airflow/logs	Logs internos de Airflow

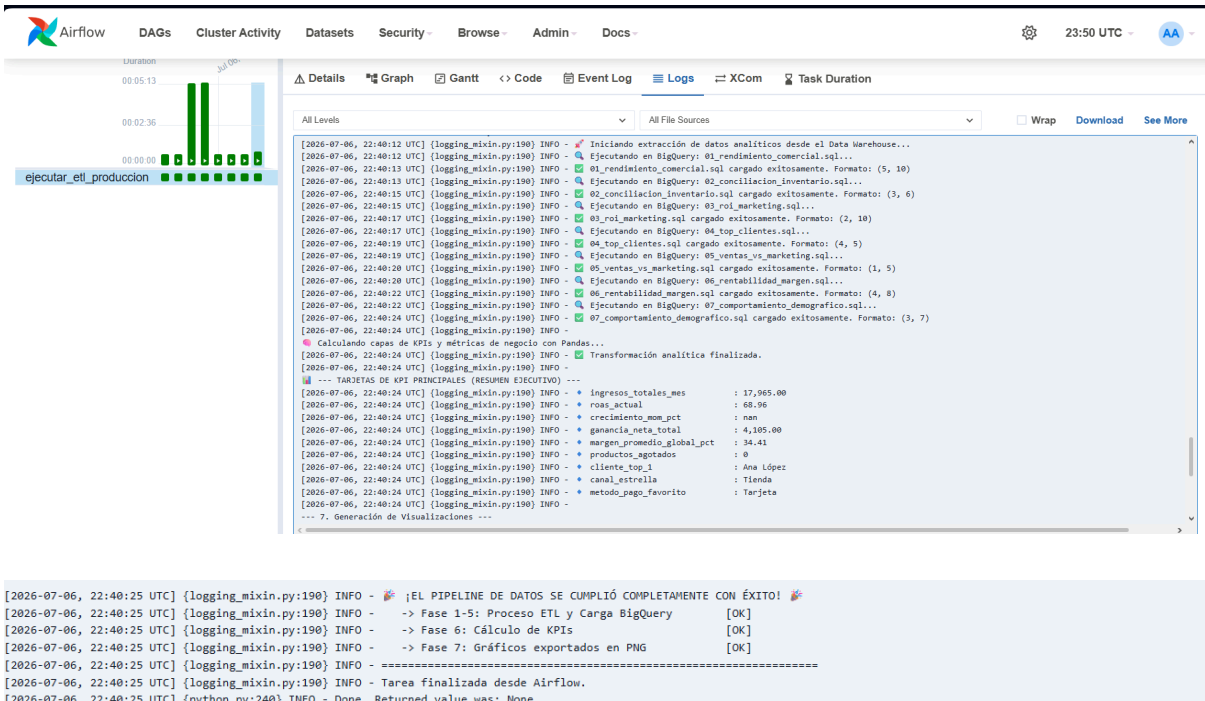
El uso de Docker garantiza que Airflow pueda acceder a las fuentes, código, consultas y reportes mediante rutas controladas, reduciendo problemas de configuración entre entornos.

18. Evidencia de ejecución

La ejecución del pipeline fue validada mediante los logs de Airflow. En la ejecución final se confirmó:

Fase	Estado
Extracción de productos, ventas, clientes, inventario y marketing	Exitosa
Transformación y limpieza de fuentes	Exitosa
Construcción de dimensiones	Exitosa
Construcción de tablas de hechos	Exitosa
Carga del Data Warehouse local en SQLite	Exitosa
Migración de tablas hacia BigQuery	Exitosa
Ejecución de 7 consultas SQL analíticas	Exitosa
Cálculo de KPIs con Pandas	Exitoso
Generación de dashboards PNG	Exitosa

La ejecución final muestra que las consultas SQL analíticas se ejecutaron correctamente en BigQuery y que posteriormente se calcularon KPIs como ingresos totales, ROAS, ganancia neta, margen promedio, cliente principal, canal estrella y método de pago favorito.



18.1 Tablas migradas a BigQuery

Tipo	Tablas
Dimensiones	dim_tiempo, dim_cliente, dim_producto, dim_sucursal, dim_canal, dim_bodega, dim_metodo_pago, dim_campana
Hechos	fact_ventas, fact_inventario, fact_movimientos_inventario, fact_marketing

18.2 Archivos gráficos generados

Archivo	Descripción
01_tendencia_ventas_roas.png	Evolución de ingresos y ROAS
02_rentabilidad_top5.png	Productos más rentables

<code>03_distribucion_canal.png</code>	Distribución de ingresos por canal
<code>04_ranking_clientes.png</code>	Ranking de clientes
<code>05_inventario_rotacion.png</code>	Rotación de inventario

19. Reproducibilidad del proyecto

El proyecto fue diseñado para ejecutarse nuevamente sin intervención manual sobre las fuentes originales.

19.1 Ejecución del entorno

Desde la carpeta `docker-airflow`:

```
docker compose up -d
```

Luego se ingresa a Airflow:

URL: `http://localhost:8080`

19.2 Ejecución del pipeline

1. Activar el DAG `pipeline_datacommerce_etl_v1`.
2. Ejecutar manualmente o esperar la programación diaria.
3. Revisar los logs de la tarea `ejecutar_etl_produccion`.
4. Validar la generación del Data Warehouse local.
5. Revisar la carpeta `dashboards/` con los archivos PNG generados.

19.3 Elementos regenerables

Elemento	Estado
<code>datacommerce_dw.db</code>	Se regenera con el pipeline
Dashboards PNG	Se generan automáticamente

Logs de Airflow	Se generan en cada ejecución
Tablas BigQuery	Se reemplazan con <code>WRITE_TRUNCATE</code>

20. Seguridad y control de versiones

Se configuró `.gitignore` para evitar subir archivos sensibles, temporales o generados automáticamente.

20.1 Archivos que no deben subirse a GitHub

Archivo / carpeta	Motivo
<code>data/proyectofinalg2-92ad91289d8e.json</code>	Credencial privada de Google Cloud
<code>data/datacommerce_dw.db</code>	Base generada automáticamente
<code>docker-airflow/logs/</code>	Logs generados por Airflow
<code>docker-airflow/.env</code>	Variables locales de entorno
<code>.venv/</code>	Entorno virtual local
<code>__pycache__/</code>	Caché de Python
<code>*.log</code>	Archivos de logs

20.2 Archivos que sí forman parte del repositorio

Carpeta / archivo	Motivo
<code>src/</code>	Código fuente del pipeline
<code>dags/</code>	DAG de Airflow
<code>sql/</code>	Consultas analíticas
<code>notebooks/</code>	Evidencia exploratoria por fases
<code>data/ventas.csv</code>	Fuente de ventas
<code>data/productos.xlsx</code>	Fuente de productos

<code>data/clientes.json</code>	Fuente de clientes
<code>data/inventario.db</code>	Fuente de inventario
<code>data/api_marketing_response.json</code>	Fuente de marketing simulada
<code>docker-airflow/docker-compose.yaml</code>	Configuración de Airflow en Docker
<code>README.md</code>	Guía de ejecución
<code>requirements.txt</code>	Dependencias del proyecto

21. Limitaciones

Limitación	Descripción
Dataset pequeño	Los datos utilizados son de muestra y no representan una operación real de gran escala
API simulada	La fuente de marketing fue representada mediante una API REST simulada / JSON
Histórico limitado	Algunos KPIs temporales no pueden calcular comparación mensual completa
<code>crecimiento_mom_pct</code> con valor NaN	Se debe a falta de un período anterior suficiente para comparar
Informe empresarial separado	La interpretación gerencial profunda se desarrollará como documento complementario
Entorno local	La ejecución depende de Docker, credenciales de Google Cloud y configuración local

Estas limitaciones no afectan la validez técnica del pipeline, pero sí delimitan el alcance del análisis empresarial.

22. Conclusiones técnicas

1. Se diseñó e implementó una plataforma de datos funcional para integrar fuentes heterogéneas de Data Commerce GT.
2. Se aplicaron reglas de calidad de datos mediante código, evitando modificaciones manuales sobre las fuentes originales.
3. Se diseñó un modelo relacional conceptual que representa las entidades operativas del negocio.
4. Se implementó un modelo dimensional tipo galaxia, adecuado para analizar ventas, inventario, movimientos y marketing.
5. Se construyó un Data Warehouse local en SQLite con dimensiones y tablas de hechos.
6. Se migraron las tablas del Data Warehouse hacia Google BigQuery usando el cliente oficial de Google.
7. Se ejecutaron consultas SQL analíticas para preparar DataFrames orientados a negocio.
8. Se calcularon KPIs comerciales, financieros, operativos y ejecutivos con Python y Pandas.
9. Se generaron visualizaciones automáticas para apoyar conclusiones de negocio.
10. Se orquestó el proceso completo con Apache Airflow en Docker, logrando una ejecución reproducible y controlada.

En conjunto, el proyecto cumple con los requerimientos del Capstone al integrar datos, aplicar calidad, modelar información, cargar un Data Warehouse, ejecutar SQL analítico, construir KPIs y presentar visualizaciones para apoyar la toma de decisiones.

23. Anexos

23.1 Estructura final del repositorio

PROYECTO_FINAL_GRUPO2/

```
|
|— .github/
|   |— workflows/
|
|— dags/
|   |— dag_pipeline.py
|
|— dashboards/
|   |— 01_tendencia_ventas_roas.png
|   |— 02_rentabilidad_top5.png
```

- ├── 03_distribucion_canal.png
- ├── 04_ranking_clientes.png
- ├── 05_inventario_rotacion.png
- ├── data/
 - ├── ventas.csv
 - ├── productos.xlsx
 - ├── clientes.json
 - ├── inventario.db
 - └── api_marketing_response.json
- ├── docker-airflow/
 - ├── docker-compose.yaml
 - ├── dags/
 - ├── plugins/
 - └── logs/ # Ignorado por Git
- ├── docs/
 - ├── arquitectura/
 - ├── modelos/
 - ├── diccionario_datos/
 - └── informe/
- ├── notebooks/
 - ├── 1_extraccion.ipynb
 - ├── 2_transformacion.ipynb
 - ├── 3_carga.ipynb
 - └── 4_analisis.ipynb
- ├── sql/
 - ├── 01_rendimiento_comercial.sql
 - ├── 02_conciliacion_inventario.sql
 - ├── 03_roi_marketing.sql
 - ├── 04_top_clientes.sql
 - ├── 05_ventas_vs_marketing.sql
 - ├── 06_rentabilidad_margen.sql
 - └── 07_comportamiento_demografico.sql
- ├── src/
 - ├── pipeline_manager.py
 - ├── config.py
 - ├── extract/
 - ├── transform/
 - ├── load/
 - └── analytics/
- ├── .gitignore
- └── Dockerfile

└─ README.md
└─ requirements.txt